Iterative Training for SEM Masking: U-Net + Evaluator

Nebil Mohammed Alex Strugacz

Abstract

Scanning Electron Microscope (SEM) images of wafer cross-sections present significant challenges for reliable segmentation. Classical binarization and morphological techniques often fail due to varying structures and contrast conditions, while manual mask curation is accurate but computationally expensive and slow to scale. This paper proposes a robust approach that combines a U-Net architecture for mask proposal and a ResNet18-based Evaluator to score mask quality. By utilizing an iterative training strategy, we expand the training set with high-confidence pseudo-labels, allowing the model to scale effectively while maintaining segmentation accuracy.

1 Introduction

Reliable segmentation of Scanning Electron Microscope (SEM) images is a critical task in wafer analysis. However, it is fraught with difficulties. SEM images of wafer cross-sections are inherently hard to segment reliably. Traditional methods, such as classical binarization combined with morphological operations, often fail when optimizing for different structures and varying contrast conditions.

Conversely, while manual mask curation provides high accuracy, it is far too slow to scale to the volume of data required for modern applications. To address these limitations, we introduce a dual-model approach:

- **U-Net:** A model dedicated to proposing segmentation masks.
- **Evaluator:** A model designed to score the quality of the generated masks (Good vs. Bad).

Our strategy leverages these models in an iterative loop to progressively expand the training dataset using high-confidence pseudo-labels.

2 Prior Work

The foundation of Deep Learning-based metrology for this dataset was established by Ding and Peng [1], who proposed a "Double U-Net" strategy. Their approach sought to overcome the scarcity of labeled SEM data by training an initial U-Net on a small, manually annotated dataset, generating predictions on unlabeled data, and then *manually curating* these predictions to form a larger training set for a second, refined U-Net.

While Ding's work demonstrated that iterative retraining could improve segmentation fidelity, straightforward reproduction of this pipeline revealed significant limitations in scalability and robustness:

- **Manual Curation Bottleneck:** The "Double U-Net" method relies entirely on human experts to filter the output of the first model. This manual verification step is labor-intensive and susceptible to curator fatigue, effectively placing a hard ceiling on dataset size. Consequently, Ding's approach was limited to a single re-training iteration ($n = 2$).
- **Lack of Zero-Shot Validation:** The prior methodology presumes that a model trained on one wafer will generalize to others. However, without an automated mechanism to validate or fine-tune predictions on new distributions (e.g., unseen wafers with different contrast profiles), the pipeline remains brittle to domain shifts.

Our Contribution: We depart from the "Double U-Net" paradigm by replacing the human curator with a differentiable *Auto-Curator* (a ResNet18-based Evaluator). By determining mask quality algorithmically, we close the loop, enabling a fully autonomous iterative pipeline ($n = 4+$). This approach makes a dent in the scalability barrier by allowing us to double the dataset size without human intervention.

3 Dataset Overview

The initial dataset consists of manually curated Scanning Electron Microscope (SEM) images of wafer cross-sections used for segmentation of etched structures. All images are grayscale SEM micrographs stored as TIFF files, with corresponding binary ground-truth masks indicating etched foreground versus background material.

Each sample consists of a paired image and mask with matching filename stems. The manually labeled dataset contains a total of 867 image–mask pairs that span five types of etched structures. Images exhibit substantial variability in contrast, geometry, and noise patterns, making segmentation challenging using classical thresholding and morphological methods.

3.1 Structures and Wafers

The initial manually curated dataset was based on the training data used for Shuhan Ding's first U-Net segmentation model [1], providing a consistent baseline for comparison while allowing further extension through iterative learning. The dataset includes the following etched structure types: DTGAP, ETE, TETE, TGAP, and UL. Each structure exhibits distinct geometric profiles and contrast characteristics in SEM cross-sections.

Table 1: Composition of the manually curated initial dataset.

Structure	**Manual Count**
DTGAP	193
ETE	205
TETE	39
TGAP	220
UL	210
Total	**867**

Data were collected across multiple wafers following the wafer indexing convention used in prior work [2], where wafers are labeled as A = 1, B = 2, C = 3, D = 4, E = 6, F = 7, G = 8, and H = 9. Wafer 5 is not included in this dataset.

Certain structure types were excluded from the initial dataset. In particular, TCL and FLG were omitted due to duplicate images, and TTGAP was not available in the curated ground-truth labeling. These exclusions were applied to avoid bias and data leakage during training.

4 Methodology

4.1 The Evaluator Model

The Evaluator serves as a quality control mechanism, classifying input pairs of raw SEM images and candidate masks as either "Good" or "Bad."

4.1.1 Dataset Construction

The dataset for the Evaluator was constructed using distinct sources to ensure robustness:

- **Curated Set (Good):** Trusted masks obtained from manual labeling (specifically referencing Shuhan's labeling).
- **Bad Mask Set (Bad):** Masks identified as noisy or clearly incorrect from the manual labeling process.

Scripts were utilized to organize these into `1_Good/` and `0_Bad/` directories containing aligned images and masks.

4.1.2 Architecture and Training

The Evaluator utilizes a **ResNet-18** architecture comprising **11.18 million trainable parameters**. To accommodate the input requirements, the first convolutional layer was modified to accept 4 channels (3 RGB channels + 1 binary mask channel) instead of the standard 3. The final Fully Connected (FC) layer was replaced to output a single logit representing the binary validity of the mask.

Training details are as follows:

- **Input Resolution:** Resized to 224×224 pixels.
- **Data Split:** 80/20 train-validation split.
- **Hyperparameters:** Batch size of 16; Learning Rate (LR) of $1e-4$ optimized via Adam; 40 epochs.
- **Loss Function:** `BCEWithLogitsLoss` with a dynamic positive weight (`pos_weight`) to counteract class imbalance.
- **Decision Threshold:** A conservative threshold ($T = 0.74$) was selected to minimize false positives.

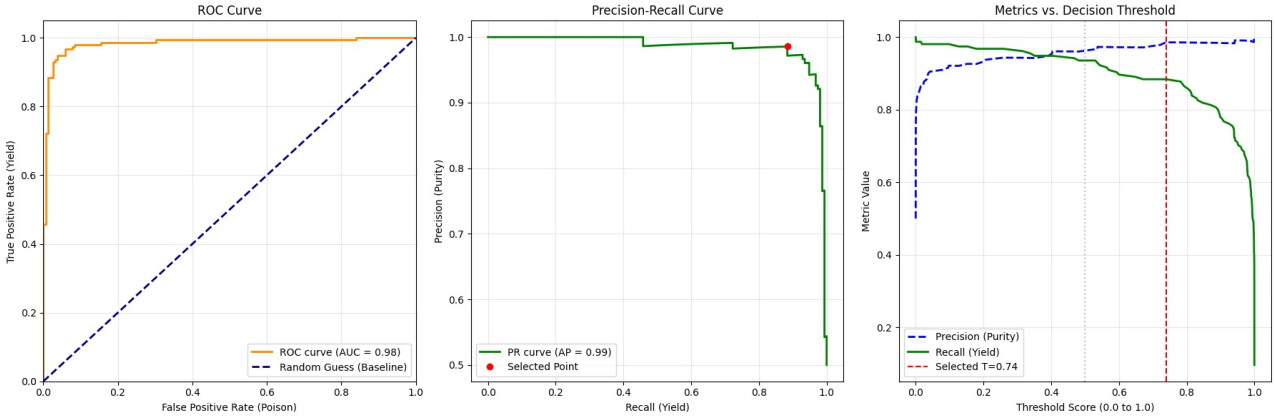


Figure 1: **Evaluator Performance Metrics.** (Left) Receiver Operating Characteristic (ROC) curve with an AUC of 0.98, indicating near-perfect separability between high-quality and low-quality masks. (Right) Precision-Recall (PR) curve with an Average Precision (AP) of 0.99. The red marker indicates the selected operating threshold ($T = 0.74$), chosen to maximize dataset yield while constraining the False Positive Rate to prevent label poisoning in the iterative loop.

ROC Curve. The ROC curve plots True Positive Rate (TPR) vs. False Positive Rate (FPR) as the decision threshold varies. AUC summarizes ranking quality: higher AUC means the evaluator more consistently scores good masks above bad masks.

ROC Analysis: Separability and Robustness. The ROC curve visualizes the trade-off between the True Positive Rate (Recall) and the False Positive Rate (FPR). Our model achieves an Area Under the Curve (AUC) of **0.98**, characterized by a steep vertical rise in the top-left quadrant. This steepness indicates that the Evaluator can retrieve the vast majority of valid masks before incurring any significant false positives. In the context of our iterative pipeline, this high separability is critical; it ensures that the "Good" and "Bad" distributions are distinct enough that slight shifts in the decision threshold do not result in catastrophic acceptance of noise.

Precision-Recall Analysis: The Autonomy Constraint. While ROC allows for model comparison, the Precision-Recall (PR) curve is the primary metric for deployment, particularly given the class imbalance in the unlabeled pool.

- **Precision (Purity):** The probability that a mask classified as "Good" is actually valid.
- **Recall (Yield):** The percentage of total valid masks in the pool that we successfully mine.

As shown in Figure 1, we operate in the high-precision regime (top-left of the PR curve). We prioritized **Dataset Purity** over Yield. Accepting a bad mask (False Positive) introduces "label poison" that reinforces errors in the next U-Net generation (Model Drift). Conversely, rejecting a good mask (False Negative) merely slows dataset growth but preserves integrity. Consequently, we selected a conservative threshold ($T = 0.74$) that maintains Precision > 0.95, ensuring the autonomous loop remains stable even if it means discarding some valid data.

4.2 The U-Net Model

The U-Net is responsible for the pixel-wise segmentation task.

4.2.1 Architecture

We employed the same Unet architecture as Shuhan, a highly efficient custom U-Net (`unet.Unet`) with a 4-stage encoder-decoder depth and a channel configuration of (32, 64, 128, 256). The model contains only 0.64 million (640,513) trainable parameters. This lightweight design is significantly smaller than standard U-Net implementations and helps not to overfit on the initial small dataset, and to enable rapid inference across the 4,000-image unlabeled pool.

4.2.2 Training Configuration

The base training dataset consisted of paired images and labels with matching file stems. The `SEMDataset` class handled paired loading and normalization.

- **Loss Function:** Hybrid BCEDiceLoss (combining Binary Cross Entropy and Dice Loss) to address boundary sharpness.
- **Optimization:** Adam optimizer with a learning rate of $1e-4$.
- **Hyperparameters:** Batch size of 8; approximately 30 epochs per stage.
- **Data Split:** 80/20 random split into training and validation sets.

Specific datasets (TCL and FLG) were initially excluded due to duplicate/quality issues, and TTGAP was absent from the curated set.

4.3 Iterative Training Strategy

The core contribution of this work is the iterative training loop designed to expand the dataset without manual intervention:

1. **Train U-Net:** Train the U-Net on the current available dataset.
2. **Inference:** Run inference on the full unlabeled SEM pool ($N \approx 4000$) to generate candidate masks.
3. **Evaluation:** Pass the candidate masks through the ResNet-18 Evaluator.
4. **Filtering:** Retain only predictions with a confidence score > 0.74.
5. **Dataset Expansion:** Merge high-confidence pseudo-labels into the training set and repeat for $n = 4$ stages.

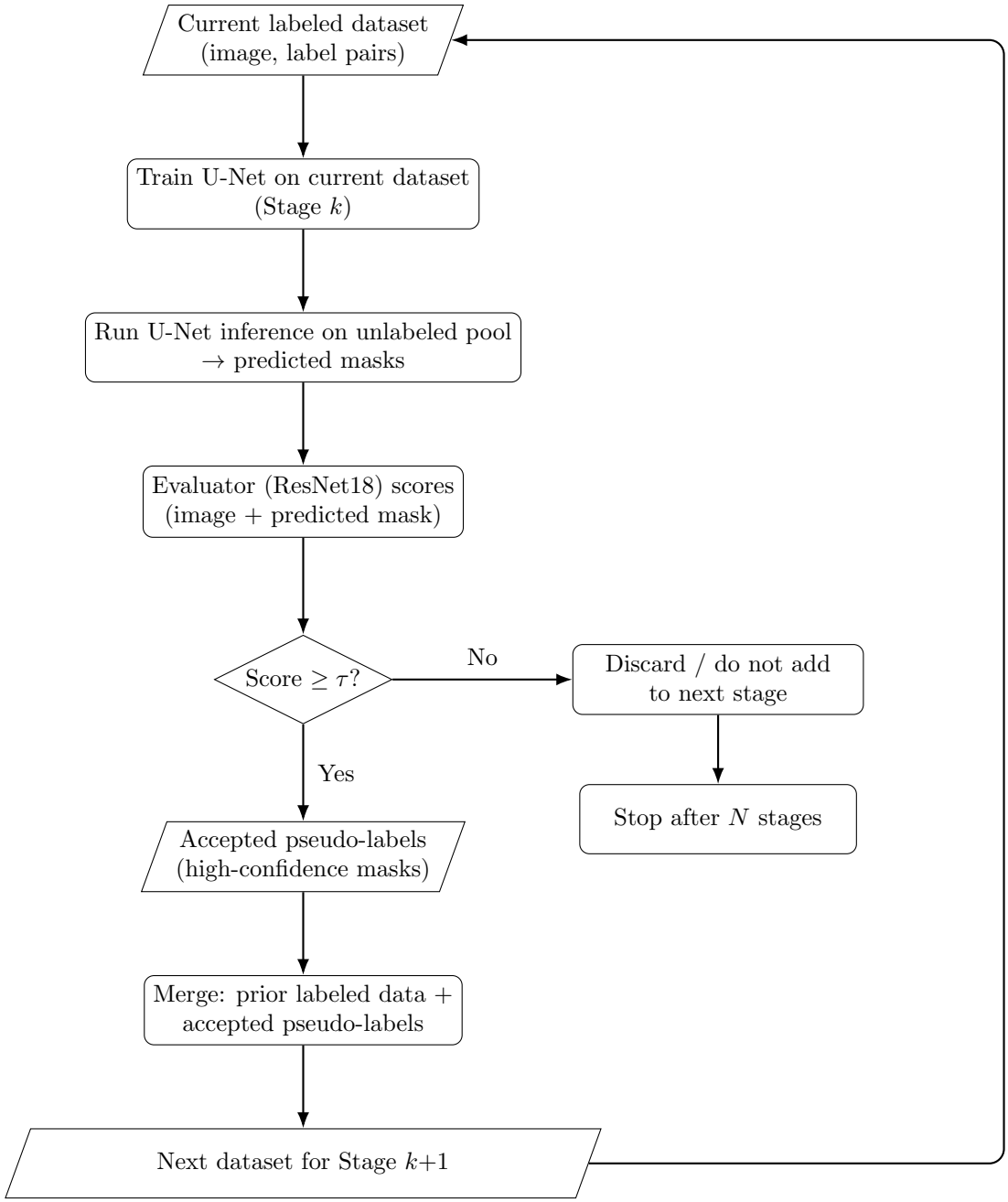


Figure 2: Iterative training loop: a U-Net proposes masks on unlabeled SEM images, an Evaluator filters high-confidence pseudo-labels, and accepted predictions are merged into the labeled dataset for the next stage.

5 Results

5.1 Evaluator Performance and Training Efficiency

The Evaluator demonstrated high performance during validation, achieving an Area Under the Curve (AUC) of 0.98 on the ROC curve and an Average Precision (AP) of 0.99 on the Precision–Recall curve. Based on these metrics, a decision threshold of 0.74 was selected to strictly optimize for precision (purity), prioritizing the exclusion of false positives over the maximization of yield.

We initially explored data augmentation strategies, including horizontal flips and synthetic label corruption (dilating "Bad" masks), to improve the Evaluator's robustness. However, these techniques introduced unrealistic artifacts that degraded the validation performance. Similarly, training on the

raw, unbalanced dataset biased the model toward the majority class. Consequently, we utilized a strictly balanced dataset ($N = 867$ pairs) for the final Evaluator.

Training was performed on consumer-grade hardware (NVIDIA RTX 3050). The Evaluator required approximately 40 minutes to converge. The full iterative U-Net pipeline ($n = 4$) required 12–16 hours to complete, with the majority of computational time dominated by the inference step across the 4,000-image unlabeled pool.

5.2 Dataset Scalability

The iterative pipeline demonstrated significant autonomous scaling. As shown in Table 2, the training set grew from a manual baseline of 867 images to a final size of 1,593 images—an 83.7% increase achieved with zero additional human labeling. This represents a 1.84x scaling factor on data availability.

The system exhibited rapid convergence, with the majority of "easy" examples mined in Stage 2. This was followed by a stabilizing plateau in Stages 3 and 4, confirming that the Evaluator successfully rejected outlier data rather than allowing the dataset to drift indefinitely.

Table 2: **Methodology Scalability.** Comparison of the manual baseline versus the final auto-curated result ($n = 4$). The pipeline nearly doubled the dataset size without human intervention.

Metric	Manual (Old)	Automated (New)	Improvement
Dataset Size	867 Images	1,593 Images	+726 Images
Scaling Factor	1.0x (Ref)	1.84x	**83.7% Increase**
Human Labor	High (Manual)	None	Fully Autonomous

5.3 Structural Generalization

The auto-curator did not merely replicate the class distribution of the manual baseline but actively expanded coverage for complex structures. Table 3 highlights a massive expansion in ETE structures (+396 images) and TGAP (+169 images). Notably, the model successfully generalized to UL structures ($N = 284$), expanding the existing pool by 35% without manual intervention.

Table 3: **Structural Generalization.** The Auto-Curator successfully expanded the dataset across all major structures, with significant gains in ETE and TGAP geometries.

Structure	Manual Count	Auto Count	Net Change	Status
DTGAP	193	218	+25	Expanded
ETE	205	601	+396	**Major Expansion**
TETE	39	101	+62	Expanded
TGAP	220	389	+169	Expanded
UL	210	284	+74	Expanded

5.4 Qualitative Segmentation Analysis

Figure 3 compares the manual "Gold Standard" masks against those generated by our Iteration-4 Auto-Curator. The auto-generated masks exhibit boundary adherence and topological cleanliness indistinguishable from the manual baseline.

Conversely, Figure 4 demonstrates the Evaluator's robustness. The system correctly identified and rejected incomplete segmentations and background noise, ensuring that only high-fidelity masks

entered the training loop. While effective, the strict threshold occasionally resulted in the rejection of potentially usable masks, highlighting a trade-off between dataset purity and recall.

6 Limitations, Failure Modes, and Negative Results

While the iterative U-Net + Evaluator pipeline substantially scales the labeled dataset, we observed failure modes that limit reliability and generalization. These issues matter because the loop amplifies bias: overly strict filtering reduces yield, while false positives risk contaminating later stages.

6.1 Evaluator Does Not Generalize Reliably to Unseen Structures

The Evaluator performs worse on *unseen* structures/wafers than on the curated validation split, likely due to learned priors tied to the initial labeled distribution. Under shifts in geometry, contrast, or wafer-specific imaging characteristics, it often misclassifies plausible masks as "Bad," despite strong in-split ROC/PR metrics.

6.2 Lack of Ground Truth on Other Wafers Prevents Rigorous Verification

Most unlabeled wafers lack ground-truth masks, limiting our ability to quantify cross-wafer generalization and to separate true rejections of incorrect masks from over-rejection caused by distribution shift.

6.3 Low Operating-Point Accuracy and Threshold Sensitivity

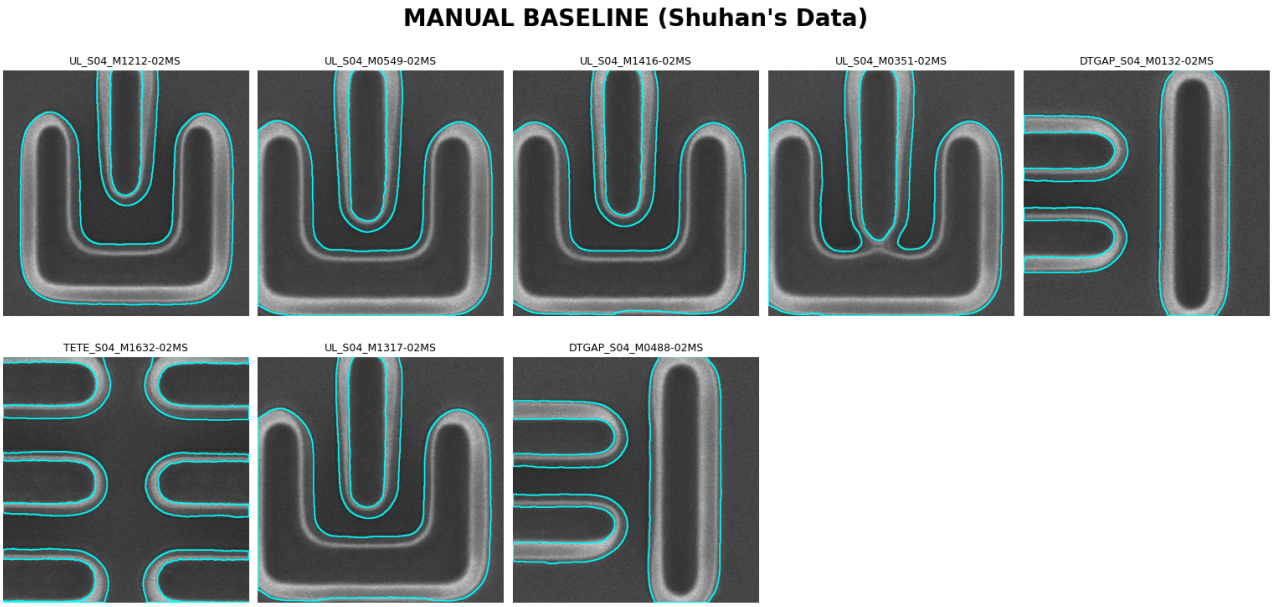
At the chosen operating point, raw Evaluator accuracy is modest ($\sim$76%), partly because we use a conservative threshold τ to minimize false positives. However, performance is sensitive to τ: high τ reduces dataset growth via over-rejection, while low τ increases the risk of admitting low-quality pseudo-labels and drift.

6.4 Over-Rejection of "Deformed but Correct" Segmentation

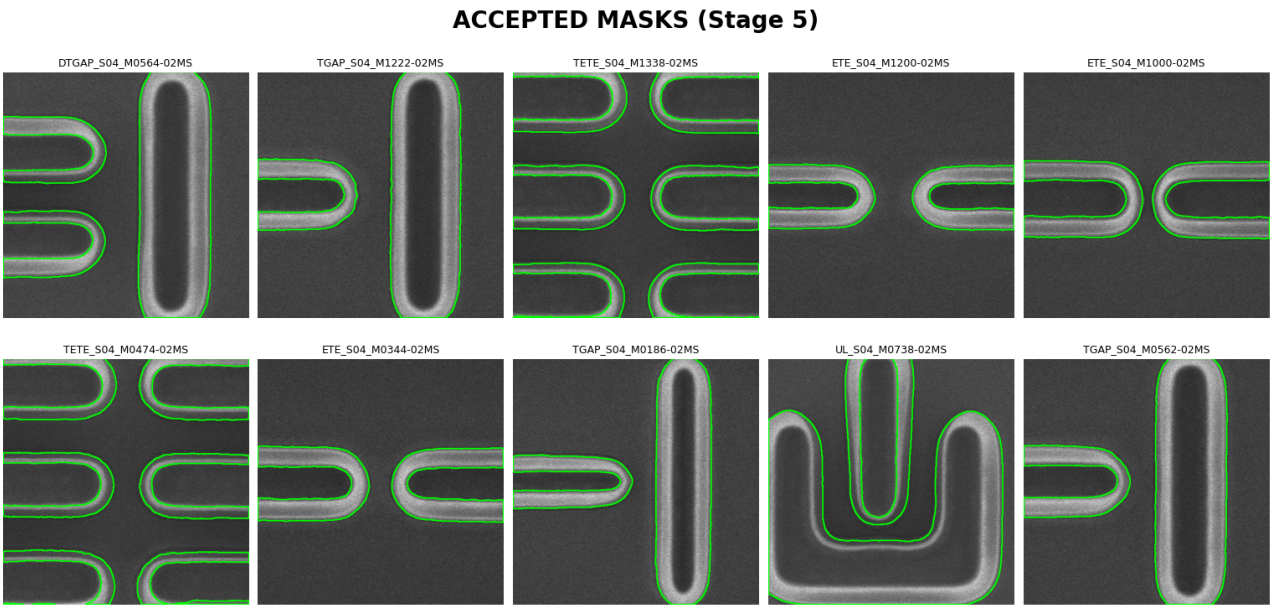
The Evaluator also rejects masks that are semantically correct but deviate slightly from the curated style (e.g., small boundary shifts, local warps, or atypical yet valid topology), suggesting it penalizes non-canonical shapes even when the segmentation is usable.

6.5 Implications and Practical Takeaways

Overall, the system behaves as a high-precision, low-recall auto-curator, with over-rejection as the dominant failure mode under distribution shift and for non-canonical masks. Improving robustness likely requires wafer-diverse ground-truth evaluation and better generalization in the Evaluator.



(a) **Manual Baseline (Prior Work).** Manually curated masks ($N = 867$).



(b) **Auto-Curated Results (Ours).** Masks generated autonomously ($N = 1,593$).

Figure 3: **Qualitative Parity Analysis.** A comparison of (a) manually labeled masks versus (b) our auto-generated masks. The automated pipeline produces contours with identical topological fidelity to human curation, demonstrating that the dataset expansion maintains high quality.

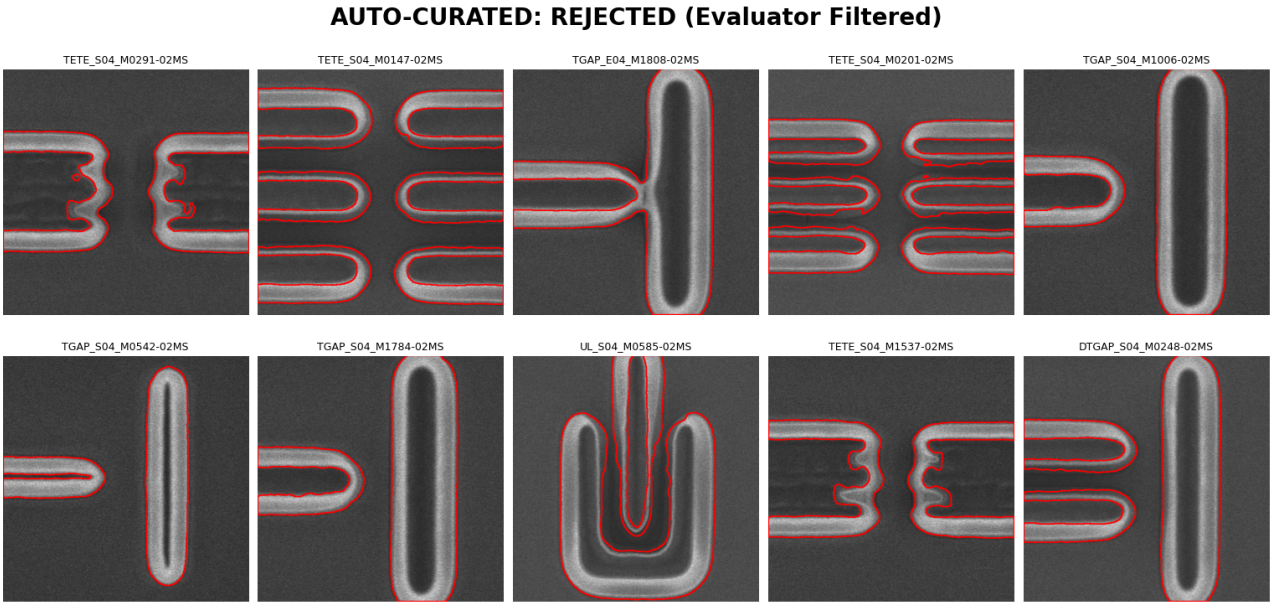


Figure 4: **Evaluator Rejection Samples.** Examples of candidate masks rejected by the Evaluator. The system acts as a safety filter against model drift but is very strict, occasionally rejecting usable labels to maintain dataset purity.

7 Conclusion and Next Steps

This work presents a fully autonomous iterative training pipeline for SEM cross-section segmentation that replaces the manual curation step in prior "Double U-Net" workflows with a learned mask-quality *Auto-Curator*. Starting from a manually labeled baseline of 867 image–mask pairs across five etched structures (DTGAP, ETE, TETE, TGAP, UL), we trained a lightweight U-Net to propose masks on an unlabeled pool ($N \approx 4000$) and used a ResNet-18 Evaluator (4-channel image+mask input) to filter high-confidence pseudo-labels. Across four stages, the pipeline expanded the labeled set to 1,593 samples (+83.7%) with zero additional human labeling, while preserving qualitative parity with the manual "Gold Standard" contours (Figure 3). In-distribution validation suggests the Evaluator is a strong ranking model (ROC AUC = 0.98, PR AP = 0.99), and a conservative operating threshold ($T = 0.74$) enabled stable dataset growth by prioritizing precision and minimizing the risk of label poisoning in downstream stages.

At the same time, our experiments highlight that the current system behaves as a *high-precision, low-recall* curator: the dominant failure mode is over-rejection under distribution shift (unseen structures/wafers) and for masks that are semantically correct but "non-canonical" relative to the curated style. This trade-off is acceptable for safe autonomous bootstrapping, but it becomes a bottleneck when the goal is maximum yield or robust cross-wafer generalization. Overall, the key takeaway is that coupling a strong segmenter with an explicit, conservative quality gate can scale labeled SEM data reliably, but the curator's generalization ultimately determines how far the loop can be pushed.

Potential next steps include:

1. **Broaden coverage (FLG/TCL and additional wafers):** incorporate the excluded structure sets and expand beyond the current wafer distribution to test scalability on new regimes.

2. **Add wafer-diverse ground truth:** curate a small, representative labeled subset on additional wafers to quantify true cross-wafer generalization and separate valid rejection from distribution-shift over-rejection.

3. **Improve Evaluator robustness and acceptance logic:** train the Evaluator with more structure/wafer diversity, explore domain-robust augmentations, and reduce reliance on a single

hard threshold via calibration, uncertainty-aware acceptance (e.g., graded confidence bins), or wafer/structure-adaptive thresholds.

These steps aim to move the pipeline from a conservative auto-curator toward a more robust and higher-yield autonomous labeling system while maintaining the stability required for iterative retraining.

A Appendix

This appendix contains the core code used for the iterative training loop described in the paper: (1) U-Net stage training, (2) U-Net inference on unlabeled SEMs, (3) Evaluator scoring (Good/Bad), and (4) automatic dataset growth across stages.

`code/unet/iterative_training.py`

```
import os
import json
import shutil
from pathlib import Path
from typing import Literal, List, Dict

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
from PIL import Image

# U-Net stage training + inference
from unet_training import train_stage
from unet_inference import infer_all


def convert_path(path: str) -> str:
    return path.replace("\\", "/") if "\\" in path else path


# ---------- EVALUATOR DATASET & MODEL ----------

class EvaluatorInferenceDataset(Dataset):
    """
    Dataset for running the evaluator on predicted masks.
    Produces 4-channel input = RGB image (3ch) + mask (1ch).
    """
    def __init__(self, image_root: str, mask_root: str):
        self.samples: List[tuple[str, str, str]] = []
        valid_exts = (".tif", ".tiff", ".png", ".jpg", ".jpeg")

        masks_by_stem: Dict[str, str] = {}
        for root, _, files in os.walk(mask_root):
            for f in files:
                if f.lower().endswith(valid_exts):
                    stem = os.path.splitext(f)[0]
                    masks_by_stem[stem] = os.path.join(root, f)

        for root, _, files in os.walk(image_root):
            for f in files:
                if f.lower().endswith(valid_exts):
                    stem = os.path.splitext(f)[0]
                    if stem in masks_by_stem:
                        img_path = os.path.join(root, f)
```

```
                    msk_path = masks_by_stem[stem]
                    self.samples.append((img_path, msk_path, f))

        self.resize = transforms.Resize((224, 224))
        self.to_tensor = transforms.ToTensor()

    def __len__(self) -> int:
        return len(self.samples)

    def __getitem__(self, idx):
        img_path, mask_path, fname = self.samples[idx]
        image = Image.open(img_path).convert("RGB")
        mask = Image.open(mask_path).convert("L")

        image = self.resize(image)
        mask = self.resize(mask)

        img_tensor = self.to_tensor(image) # (3,H,W)
        msk_tensor = self.to_tensor(mask) # (1,H,W)
        inp = torch.cat([img_tensor, msk_tensor], dim=0) # (4,H,W)
        return inp, fname


def get_evaluator_model() -> nn.Module:
    model = models.resnet18(weights=None)
    model.conv1 = nn.Conv2d(4, 64, kernel_size=7, stride=2, padding=3, bias=False)
    model.fc = nn.Linear(model.fc.in_features, 1)
    return model


def classify_good_bad(
    image_dir: str,
    mask_dir: str,
    evaluator_model_path: str,
    device: Literal["cpu", "cuda"] = "cuda",
    batch_size: int = 16,
    threshold: float = 0.7392,
):
    device_obj = torch.device(device if torch.cuda.is_available() else "cpu")

    dataset = EvaluatorInferenceDataset(image_dir, mask_dir)
    if len(dataset) == 0:
        return {}, {}

    loader = DataLoader(dataset, batch_size=batch_size, shuffle=False)

    model = get_evaluator_model().to(device_obj)
    state = torch.load(evaluator_model_path, map_location=device_obj)
    model.load_state_dict(state)
    model.eval()

    labels: Dict[str, int] = {}
    raw_probs: Dict[str, float] = {}

    with torch.no_grad():
        for inputs, fnames in loader:
            inputs = inputs.to(device_obj)
            logits = model(inputs).squeeze(1)
            probs = torch.sigmoid(logits).cpu().numpy()

            for fname, p in zip(fnames, probs):
                labels[fname] = int(p >= threshold)
                raw_probs[fname] = float(p)
```

```python
    return labels, raw_probs


# ---------- UNION DATASET BUILDER ----------

def build_union_dataset(
    prev_data_root: Path,
    unlabeled_images_root: Path,
    preds_root: Path,
    keep_names: List[str],
    out_data_root: Path,
) -> None:
    """
    Creates a new dataset folder combining:
      (1) all prior labeled data (prev_data_root)
      (2) evaluator-accepted pseudo-labels (keep_names) from the unlabeled pool
    """
    out_img_dir = out_data_root / "image"
    out_lbl_dir = out_data_root / "label"
    out_img_dir.mkdir(parents=True, exist_ok=True)
    out_lbl_dir.mkdir(parents=True, exist_ok=True)

    # Copy previous stage data (robust to image/images, label/masks naming)
    src_prev_img = prev_data_root / "image"
    src_prev_lbl = prev_data_root / "label"
    if not src_prev_img.exists():
        src_prev_img = prev_data_root / "images"
    if not src_prev_lbl.exists():
        src_prev_lbl = prev_data_root / "masks"

    if src_prev_img.exists():
        for f in os.listdir(src_prev_img):
            src_file = src_prev_img / f
            if src_file.is_file():
                shutil.copy2(src_file, out_img_dir / f)
                if (src_prev_lbl / f).exists():
                    shutil.copy2(src_prev_lbl / f, out_lbl_dir / f)

    # Map unlabeled images by filename for fast lookup
    unlabeled_map = {}
    for root, _, files in os.walk(unlabeled_images_root):
        for f in files:
            unlabeled_map[f] = Path(root) / f

    # Copy accepted pseudo-labels (image + predicted mask)
    for fname in keep_names:
        img_stem = os.path.splitext(fname)[0]

        mask_src = None
        for ext in [".png", ".tif", ".tiff", ".jpg"]:
            p = preds_root / f"{img_stem}{ext}"
            if p.exists():
                mask_src = p
                break

        if mask_src and fname in unlabeled_map:
            shutil.copy2(unlabeled_map[fname], out_img_dir / fname)
            shutil.copy2(mask_src, out_lbl_dir / mask_src.name)


# ---------- ITERATIVE TRAINING LOOP ----------
```

```python
def iterative_unet_training(
    base_data_root: str,
    unlabeled_images_root: str,
    work_root: str,
    evaluator_model_path: str,
    n_stages: int = 5,
    keep_threshold: float = 0.7392,
    device: Literal["cpu", "cuda"] = "cuda",
    channels=(32, 64, 128, 256),
    epochs: int = 30,
    lr: float = 1e-4,
    bs: int = 8,
):
    base_data_root = Path(convert_path(base_data_root)).resolve()
    unlabeled_images_root = Path(convert_path(unlabeled_images_root)).resolve()
    work_root = Path(convert_path(work_root)).resolve()
    work_root.mkdir(parents=True, exist_ok=True)

    current_data_root = base_data_root
    model_paths: List[Path] = []

    for stage in range(1, n_stages + 1):
        model_pth = work_root / f"unet_stage{stage}.pth"

        # 1) Train U-Net
        train_stage(
            data_root=str(current_data_root),
            model_output=str(model_pth),
            epochs=epochs,
            lr=lr,
            bs=bs,
            device=device,
            channels=channels,
        )
        model_paths.append(model_pth)

        if stage == n_stages:
            break

        # 2) Inference on unlabeled pool
        preds_root = work_root / f"preds_stage{stage}"
        infer_all(
            model_pth=str(model_pth),
            images_root=str(unlabeled_images_root),
            save_root=str(preds_root),
            device=device,
            channels=channels,
            thresh=0.5,
        )

        # 3) Evaluate masks
        good_bad_labels, raw_probs = classify_good_bad(
            image_dir=str(unlabeled_images_root),
            mask_dir=str(preds_root),
            evaluator_model_path=evaluator_model_path,
            device=device,
            batch_size=16,
            threshold=keep_threshold,
        )

        if not good_bad_labels:
            break
```

```
        # Save evaluator stats
        debug_json = work_root / f"stage{stage}_stats.json"
        with open(debug_json, "w", encoding="utf-8") as f:
            json.dump({"probs": raw_probs, "threshold": keep_threshold}, f, indent=2)

        keep_names = [name for name, label in good_bad_labels.items() if label == 1]

        # 4) Build next-stage dataset automatically
        next_data_root = work_root / f"data_stage{stage+1}"
        build_union_dataset(
            prev_data_root=current_data_root,
            unlabeled_images_root=unlabeled_images_root,
            preds_root=preds_root,
            keep_names=keep_names,
            out_data_root=next_data_root,
        )
        current_data_root = next_data_root

    return model_paths


if __name__ == "__main__":
    BASE_DATA_ROOT = "Iterative_dataset\\initial_dataset"
    UNLABELED_ROOT = "Iterative_dataset\\True_RAW_Data_flattened"
    WORK_ROOT = "Iterative_dataset\\iterative_runs"
    EVALUATOR_MODEL = "Iterative_dataset\\Balanced_evalutor.pth"

    iterative_unet_training(
        base_data_root=BASE_DATA_ROOT,
        unlabeled_images_root=UNLABELED_ROOT,
        work_root=WORK_ROOT,
        evaluator_model_path=EVALUATOR_MODEL,
        n_stages=5,
        keep_threshold=0.7392,
        device="cuda",
        channels=(32, 64, 128, 256),
        epochs=30,
        lr=1e-4,
        bs=8,
    )
```

code/unet/unet_training.py

```
import torch
from torch.utils.data import DataLoader, random_split
from typing import Literal
from dataset import SEMDataset
from unet import Unet
from losses import BCEDiceLoss
from datetime import datetime

def train_stage(
    data_root: str,
    model_output: str,
    epochs: int = 50,
    lr: float = 1e-4,
    bs: int = 8,
    device: Literal["cpu","cuda"] = "cuda",
    channels=(32,64,128,256),
):
    imgs, lbls = f"{data_root}/image", f"{data_root}/label"
    ds = SEMDataset(imgs, lbls)
```

```
    n_val = max(1, int(0.2 * len(ds)))
    g = torch.Generator().manual_seed(0)
    tr_ds, val_ds = random_split(ds, [len(ds) - n_val, n_val], generator=g)

    tr_ld = DataLoader(tr_ds, batch_size=bs, shuffle=True, num_workers=4)
    val_ld = DataLoader(val_ds, batch_size=bs, shuffle=False, num_workers=4)

    model = Unet(1, 1, list(channels)).to(device)
    crit = BCEDiceLoss()
    opt = torch.optim.Adam(model.parameters(), lr=lr)

    best = float("inf")
    for ep in range(epochs):
        model.train()
        for x, y in tr_ld:
            x, y = x.to(device), y.to(device)
            opt.zero_grad()
            loss = crit(model(x), y)
            loss.backward()
            opt.step()

        model.eval()
        tot = 0.0
        with torch.no_grad():
            for x, y in val_ld:
                x, y = x.to(device), y.to(device)
                tot += crit(model(x), y).item()
        avg = tot / len(val_ld)

        msg = f"[{datetime.now().isoformat(timespec='seconds')}] Epoch {ep+1}/{epochs}
            val_loss={avg:.4f}"
        print(msg, flush=True)

        if avg < best:
            best = avg
            torch.save(model.state_dict(), model_output)
            print(f"-> Saved {model_output}")
```

code/unet/unet_inference.py

```
import torch, cv2, numpy as np
from pathlib import Path
from typing import Literal
from unet import Unet

def infer_all(
    model_pth: str,
    images_root: str,
    save_root: str,
    device: Literal["cpu","cuda"] = "cuda",
    channels=(32,64,128,256),
    thresh: float = 0.5,
):
    model = Unet(1, 1, list(channels)).to(device).eval()

    state = torch.load(model_pth, map_location=device)
    state = state.get("model_state", state.get("state_dict", state))
    state = {k[7:] if k.startswith("module.") else k: v for k, v in state.items()}
    model.load_state_dict(state, strict=True)

    for tif in Path(images_root).rglob("*.tif"):
```

```
        img = cv2.imread(str(tif), cv2.IMREAD_GRAYSCALE).astype(np.float32) / 255.0
        x = torch.from_numpy(img)[None, None].to(device)
        with torch.no_grad():
            prob = torch.sigmoid(model(x))[0, 0].cpu().numpy()
        mask = (prob > thresh).astype(np.uint8) * 255

        outp = Path(save_root) / tif.relative_to(images_root)
        outp = outp.with_suffix(".png")
        outp.parent.mkdir(parents=True, exist_ok=True)
        cv2.imwrite(str(outp), mask)
```

code/unet/losses.py

```
import torch, torch.nn as nn

class BCEDiceLoss(nn.Module):
    def __init__(self):
        super().__init__()
        self.bce = nn.BCEWithLogitsLoss()

    def forward(self, logits: torch.Tensor, targets: torch.Tensor, eps: float = 1e-6):
        targets = targets.to(logits.dtype)
        bce = self.bce(logits, targets)
        p = torch.sigmoid(logits)
        dice = 1 - (2 * (p * targets).sum((2, 3)) / ((p + targets).sum((2, 3)) + eps)).mean()
        return bce + dice
```

code/unet/unet.py

```
from itertools import pairwise
import torch
import torch.nn as nn

def double_conv(in_c, out_c):
    return nn.Sequential(
        nn.Conv2d(in_c, out_c, kernel_size=3, padding=1, padding_mode="replicate"),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_c, out_c, kernel_size=3, padding=1, padding_mode="replicate"),
        nn.ReLU(inplace=True),
    )

class Encoder(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.pool_conv = nn.Sequential(nn.MaxPool2d(2), *double_conv(in_c, out_c))

    def forward(self, x):
        return self.pool_conv(x)

class Decoder(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.up = nn.ConvTranspose2d(in_c, out_c, kernel_size=2, stride=2)
        self.conv = double_conv(2 * out_c, out_c)

    def forward(self, x, prev):
        x = self.up(x)
        x = torch.concat([x, prev], dim=1)
        x = self.conv(x)
        return x
```

```python
class Unet(nn.Module):
    def __init__(self, in_channels, out_channels, channels):
        super().__init__()

        self.first = double_conv(in_channels, channels[0])

        self.encoders = nn.ModuleList()
        for in_c, out_c in pairwise(channels):
            self.encoders.append(Encoder(in_c, out_c))

        self.decoders = nn.ModuleList()
        for in_c, out_c in pairwise(channels[::-1]):
            self.decoders.append(Decoder(in_c, out_c))

        self.final = nn.Conv2d(channels[0], out_channels, kernel_size=1)

    def forward(self, x):
        x = self.first(x)

        convs = []
        for enc in self.encoders:
            convs.append(x)
            x = enc(x)

        convs.reverse()

        for i, dec in enumerate(self.decoders):
            x = dec(x, convs[i])

        x = self.final(x)
        return x
```

References

[1] Y. Peng and S. Ding, "Machine learning to guide printed electronics fabrication," Cornell University, School of Electrical and Computer Engineering, M.Eng. Design Project Report, Jan. 2023.

[2] C. L. Lau *et al.*, "Process-aware digital twins by deep learning for DUV photolithography and plasma etch," *IEEE Transactions on Semiconductor Manufacturing*, vol. 38, no. 3, pp. 634–641, Aug. 2025.